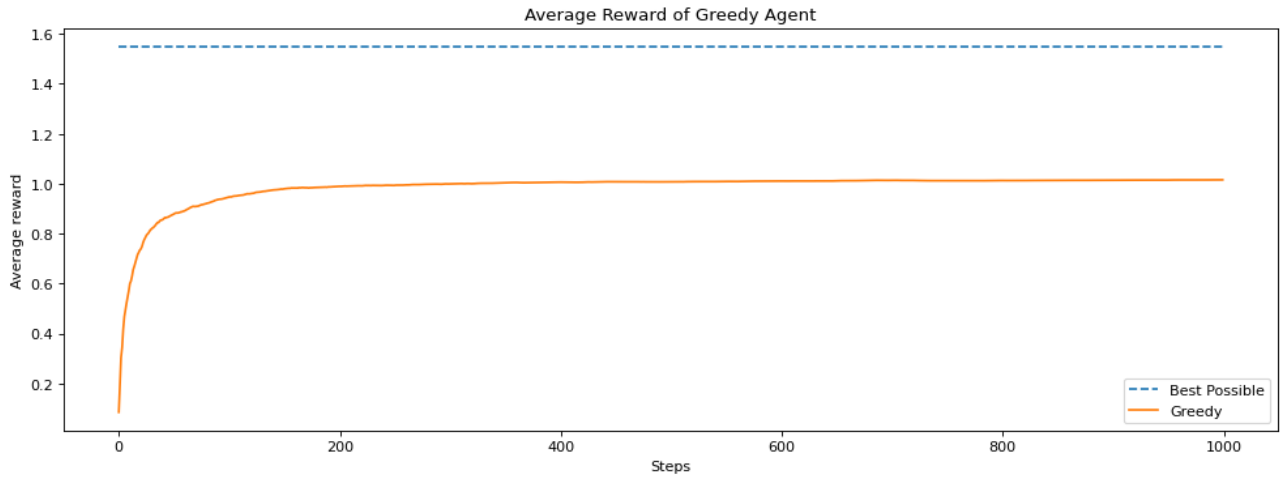
RL Lab 1 - Exploration vs Exploitation

1. Greedy Agent

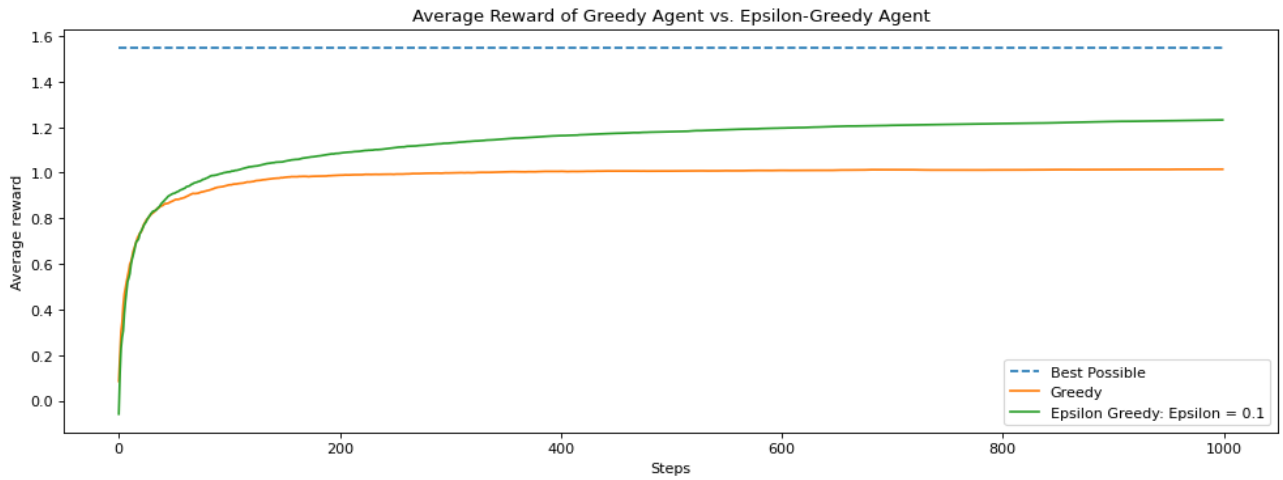
1.1 How did our agent do? Comment on the average reward graph throughout the agent's training. Is it possible for it to do better? If so, how? Please attach the graph to this answer.



Our agent did not perform very well. We can see the average reward quickly hits a plateau after some twenty steps and cannot reach higher than 1.0 averag reward. We could do better using epsilon-greedy to allow some exploration and find better actions at some steps.

2. Epsilon-Greedy Agent

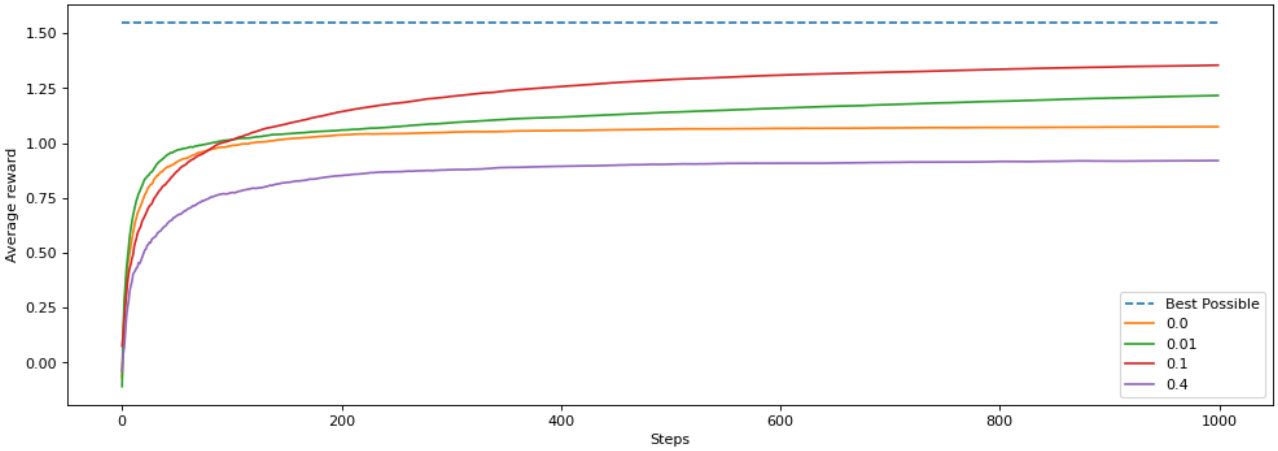
2.1 What do you notice? Explain the difference between the greedy and epsilon-greedy and comment on the average reward graph throughout the agent's training. Please attach the graph to this answer.



We can clearly see that instead of quickly hitting a plateau after a few steps, the epsilon-greedy algorithm keeps slowly finding higher average rewards paths, reaching ~1.2 average reward after 1000 steps. The difference with the greedy algorithm is that with the epsilon-greedy algorithm, we have a random chance to make a random choice, which allows to discover new better possibilities that happen at later steps, at the expense of sometimes making worse decisions.

3. Comparing values of epsilon

3.1 Why did 0.1 perform better than 0.01? Please attach the graph to this answer.



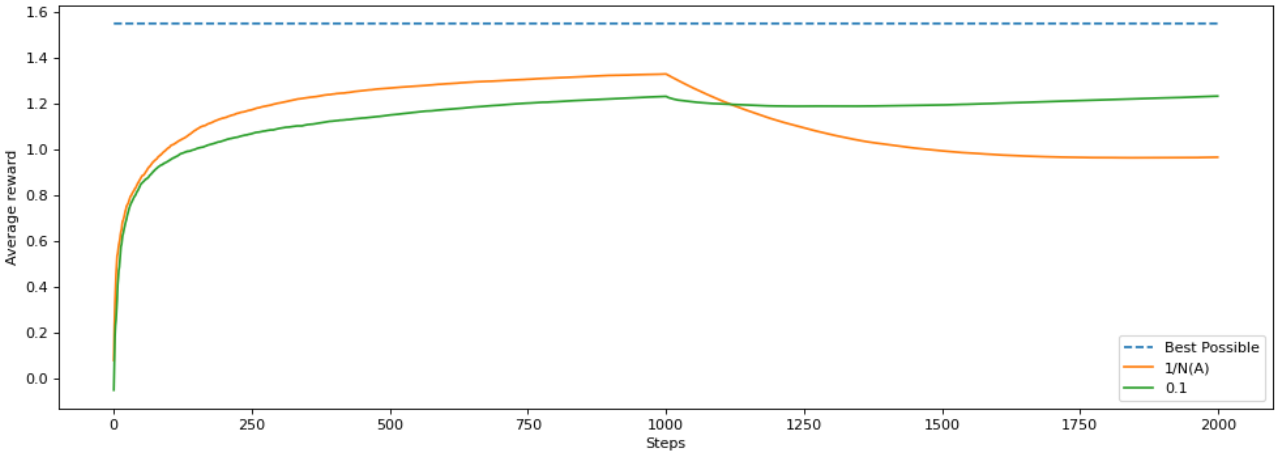
Epsilon 0.1 performed better than epsilon 0.01 because the former has a 10x higher probability of jumping. This represents many more opportunity for exploration, which allows the agent more opportunities to find the best course of action.

3.2 Why did 0.4 perform worse than 0.0 (the greedy agent)?

On the contrary, epsilon 0.4 performed worse than the greedy algorithm because at this level, the probability of a random decision is to high and the agent makes to many random choices, which are most of the time suboptimal.

4. Effects of the step size

4.1 Now the agent with a step size of 1/N(A) performed better at the start but then performed worse when the environment changed. Explain what happened. Please attach the graph to this answer.



Since the step size decreases with time using 1/N(A), after reaching a certain threshold the step size becomes too small to allow any new convergence. Adaptation changes and the algorithm is only influnced by old data. With the change of environment happening at ~1000 steps, only the constant step-size algorithm can keep learning and holds a relatively steady progress curve with a similar improvement in average reward.